FRD

Functional Requirements Document (FRD)

AWS S3-Based Data Processing Pipeline

-

FR-INGEST-001

Title

Ingest Customer and Order CSV Files from S3

Description

The system shall read customer and order data from designated S3 buckets and load them into the processing pipeline.

Preconditions

• S3 buckets `s3://adif-sdlc/sdlc_wizard/customerdata/` and `s3://adif-sdlc/sdlc_wizard/orderdata/` exist and are accessible

• CSV files conform to defined schemas

• AWS Glue has appropriate IAM permissions to read from source S3 locations

Main Flow / Functional Steps

• Read customer CSV files from `s3://adif-sdlc/sdlc_wizard/customerdata/`

• Parse customer records with schema: CustId, Name, EmailId, Region

• Read order CSV files from `s3://adif-sdlc/sdlc_wizard/orderdata/`

• Parse order records with schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

• Load parsed data into Glue processing environment

Postconditions

• Customer and order datasets are available in memory for downstream processing

• No data transformation has occurred at this stage

Assumptions

• CSV files are well-formed and delimited consistently

• File encoding is UTF-8 or compatible

• All required columns are present in source files

-

FR-CLEAN-001

Title

Remove Null Values from Customer and Order Datasets

Description

The system shall remove all records containing "Null" string values or NULL values from both customer and order datasets.

Preconditions

- Customer and order datasets have been ingested successfully
- Datasets are loaded in Glue Spark environment

Main Flow / Functional Steps

- Scan customer dataset for records containing "Null" (string) or NULL values in any column
- Remove identified records from customer dataset
- Scan order dataset for records containing "Null" (string) or NULL values in any column
- Remove identified records from order dataset

Postconditions

- Customer dataset contains no "Null" or NULL values
- Order dataset contains no "Null" or NULL values
- Record counts may be reduced

Assumptions

- "Null" refers to the literal string "Null" (case-sensitive or case-insensitive as per implementation)
- NULL refers to actual null/missing values in the data structure
-

FR-CLEAN-002

Title

Remove Duplicate Records from Customer and Order Datasets

Description

The system shall identify and remove duplicate records within each dataset independently.

Preconditions

- Null value removal (FR-CLEAN-001) has been completed
- Datasets are available in Glue Spark environment

Main Flow / Functional Steps

- Identify duplicate records in customer dataset based on all columns
- Retain one instance of each duplicate set, remove others
- Identify duplicate records in order dataset based on all columns
- Retain one instance of each duplicate set, remove others

Postconditions

- Customer dataset contains only unique records
- Order dataset contains only unique records
- Cleaned datasets are ready for cataloging

Assumptions

- Duplicates are defined as records with identical values across all columns
- No specific ordering preference for which duplicate to retain
-

FR-CATALOG-001

Title

Register Cleaned Datasets in Glue Data Catalog

Description

The system shall register cleaned customer and order datasets as tables in the Glue Data Catalog under the specified database.

Preconditions

- Data cleaning (FR-CLEAN-001, FR-CLEAN-002) is complete
- Glue Data Catalog database `gen_ai_poc_databrickscoe` exists
- IAM permissions allow table creation in the catalog

Main Flow / Functional Steps

- Create or update table `sdlc_wizard_customer` in database `gen_ai_poc_databrickscoe`
- Map customer schema: CustId, Name, EmailId, Region
- Create or update table `sdlc_wizard_order` in database `gen_ai_poc_databrickscoe`
- Map order schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Register table metadata in Glue Data Catalog

Postconditions

- Tables `sdlc_wizard_customer` and `sdlc_wizard_order` are queryable via Athena

- Schema metadata is available in Glue Data Catalog
- Tables reference cleaned data locations

Assumptions

- Database `gen_ai_poc_databrickscoe` is pre-created
- Table names do not conflict with existing tables or follow overwrite policy
-

FR-SCD2-001

Title

Implement SCD Type 2 for Order Summary Table Using Hudi

Description

The system shall maintain a slowly changing dimension (Type 2) table named `ordersummary` that joins customer and order data, tracking historical changes with active/inactive flags and timestamps.

Preconditions

- Cleaned customer and order tables are available in Glue Data Catalog
- S3 location `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/` is accessible
- Hudi library is available in Glue environment
- Table `ordersummary` is registered in database `gen_ai_poc_databrickscoe`

Main Flow / Functional Steps

- Join customer and order datasets on CustId
- For each joined record, check if it exists in current `ordersummary` table
- If record is new or changed:
- Set IsActive = true, StartDate = current timestamp, EndDate = null, OpTs = current timestamp
- Insert as new active row
- If matching active record exists with different values:
- Update existing active record: set IsActive = false, EndDate = current timestamp
- Insert new active row with updated values
- Write results to `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/` using Hudi format

Postconditions

- `ordersummary` table contains current and historical records
- Active records have IsActive = true, EndDate = null
- Closed records have IsActive = false, EndDate populated
- All records have StartDate and OpTs populated

- Data is stored in Hudi format on S3

Assumptions

- Change detection compares all business columns from customer and order join
- OpTs represents operation timestamp
- Hudi configuration supports upsert operations
- Primary key or record key is defined for Hudi operations
-

FR-SCD2-002

Title

Trigger Incremental SCD2 Updates on Customer Changes

Description

The system shall detect changes in customer data and trigger corresponding SCD Type 2 updates in the `ordersummary` table.

Preconditions

- Initial `ordersummary` table exists with baseline data
- Customer data updates are available in source S3 location
- FR-SCD2-001 logic is implemented

Main Flow / Functional Steps

- Identify new or changed customer records since last processing
- For affected CustId values, retrieve related order records
- Execute SCD2 logic (FR-SCD2-001) for affected customer-order combinations
- Update `ordersummary` table incrementally

Postconditions

- `ordersummary` reflects customer data changes
- Historical versions are preserved with proper IsActive, StartDate, EndDate, OpTs values
- Only affected records are processed (incremental update)

Assumptions

- Change detection mechanism (e.g., watermark, timestamp comparison) is in place
- Customer updates are the primary trigger; order updates follow similar pattern if applicable
- Incremental processing does not require full table scan
-

FR-AGG-001

Title

Generate Customer Aggregate Spend Table

Description

The system shall create and maintain an aggregation table `customeraggregatespend` that summarizes total spending per customer.

Preconditions

- `ordersummary` table exists and contains current data
- S3 location `s3://adif-sdlc/analytics/customeraggregatespend/` is accessible
- Table `customeraggregatespend` is registered in database `gen_ai_poc_databrickscoe`

Main Flow / Functional Steps

- Query active records from `ordersummary` (IsActive = true)
- Calculate TotalAmount per customer: sum(PricePerUnit Qty) grouped by Name
- Include Date field in output
- Write aggregated results to `s3://adif-sdlc/analytics/customeraggregatespend/`
- Register or update table schema: Name, TotalAmount, Date

Postconditions

- `customeraggregatespend` table contains one record per customer with total spend
- Table is queryable via Athena
- Data is stored at specified S3 location

Assumptions

- TotalAmount calculation uses PricePerUnit and Qty from order data
- Date represents the aggregation run date or latest transaction date
- Only active records from `ordersummary` are included in aggregation
- Aggregation is recalculated on each run (full refresh) or incrementally updated
-

FR-INFRA-001

Title

Utilize AWS Glue for ETL Processing with Spark and Hudi

Description

The system shall use AWS Glue as the ETL engine, leveraging Spark for data processing and Hudi for SCD Type 2 storage.

Preconditions

- AWS Glue service is available and configured
- Spark and Hudi libraries are accessible in Glue environment
- IAM roles grant necessary permissions for S3, Glue Catalog, and CloudWatch

Main Flow / Functional Steps

- Execute data ingestion, cleaning, and transformation logic in Glue Spark jobs
- Use Hudi for writing SCD Type 2 data to S3
- Register all tables in Glue Data Catalog
- Enable CloudWatch logging for job monitoring

Postconditions

- All ETL operations execute within AWS Glue
- Hudi format is used for `ordersummary` table
- Glue Data Catalog maintains metadata for all tables
- CloudWatch captures job logs and metrics

Assumptions

- Glue job scripts are authored in PySpark or Scala
- Hudi configuration parameters (record key, precombine field, table type) are defined
- Optional Lake Formation governance is not mandatory for initial implementation
-

FR-QUERY-001

Title

Enable Athena Querying of Cataloged Tables

Description

The system shall ensure all cataloged tables are queryable via Amazon Athena for ad-hoc analysis and reporting.

Preconditions

- Tables are registered in Glue Data Catalog (FR-CATALOG-001, FR-SCD2-001, FR-AGG-001)
- Athena has access to S3 data locations
- Athena workgroup and output location are configured

Main Flow / Functional Steps

- Verify table metadata in Glue Data Catalog is complete and accurate
- Test Athena queries against `sdlc_wizard_customer`, `sdlc_wizard_order`, `ordersummary`, and `customeraggregatespend`
- Ensure Hudi tables are readable by Athena (appropriate SerDe and input format)

Postconditions

- All tables return correct results when queried via Athena
- Users can perform SQL queries on cataloged datasets

Assumptions

- Athena supports Hudi table format with appropriate configuration
- Query performance is acceptable for expected data volumes
- No additional indexing or partitioning is required initially
-

GLOBAL ASSUMPTIONS

- All AWS services (S3, Glue, Athena, CloudWatch) are provisioned and accessible
- IAM roles and policies grant necessary permissions for cross-service operations
- CSV files arrive in source S3 buckets via external process (out of scope)
- Data volumes are within AWS Glue and Athena processing limits
- No real-time or streaming requirements; batch processing is acceptable
- Lake Formation governance is optional and not included in initial scope
- Error handling and retry logic follow AWS Glue best practices
- No specific data retention or archival policies are defined
- Schema evolution is not addressed; schemas are assumed stable
- Cost optimization and performance tuning are deferred to implementation phase